

Simulation Code Breakdown

CirceBot Simulator — Full Technical & Design Breakdown

1. High-Level Purpose of the Program

This script implements an **asynchronous robot movement simulator** that:

- Polls backend HTTP endpoints to:
 - Read `"START"` / `"STOP"` commands
 - Read paths (`/grid/path`)
 - Send status updates (`/current-values`)
- Simulates real motion in **fractional grid coordinates**
- Tracks:
 - Distance traveled
 - Remaining distance
 - Cable remaining
 - Battery drain per waypoint
- Resets internal state after reaching the final waypoint
- Sends `"STOP"` back to `/directions` upon completion
- Ensures **no PUT is sent during reset** to avoid affecting the frontend



2. Constants and Global Values

Endpoint URLs

- `HTTP_DIR_URL = "http://localhost:8765/directions"`
- `HTTP_PATH_URL = "http://localhost:8765/grid/path"`
- `HTTP_STATUS_URL = "http://localhost:8765/current-values"`

These control all I/O to the backend server.

Simulation Constants

Name	Purpose	Value
<code>SPEED_FTPS</code>	Movement rate	2 ft/s
<code>POLL_DT</code>	Simulation tick	0.5 s
<code>X_SCALE</code>	Feet per column grid step	7.5 ft
<code>Y_SCALE</code>	Feet per row step	5.16129 ft
<code>DIAG_SCALE</code>	Feet per diagonal step	9.104334 ft
<code>CABLE_MAX_FT</code>	Total available tether	300 ft
<code>EPS_FT</code>	Floating-point threshold	1e-6



3. Helper Functions

`iso_now() -> str`

Returns a UTC timestamp string.

Used for logging.



`segment_length_feet(a, b)`

Computes continuous distance between two **grid** points `{r, c}`.

Logic:

- Converts `r, c` to floats
- Computes row/column deltas
- Uses hybrid metric:
 - Diagonal as far as possible
 - Remaining horizontal/vertical with their respective scales

Used in:

- Movement interpolation
- Distance math
- Waypoint progression



`polyline_feet(points)`

Sums `segment_length_feet` across entire path.

Used only during path loading to print human-readable total distance.



`grid_to_xy_feet(r, c)`

Converts grid `(r, c)` to Cartesian `(x, y)` using scale factors.

Used when building `/current-values` payloads.



4. Class: `Simulator`

The core simulation engine.

5. Method: `reset_state()`

Resets **all local runtime variables**, including:

Variable	Meaning
<code>self.path</code>	Current waypoint list
<code>self.curr</code>	Current fractional position
<code>self.next_idx</code>	Next waypoint index
<code>self.running</code>	Whether simulator is in <code>"START"</code> state
<code>self.prev_running</code>	Tracks transitions from START→STOP
<code>self.percent_batt</code>	Battery percentage (drops 1 per waypoint)
<code>self.cable_remaining_ft</code>	Remaining cable length
<code>self.distance_traveled_ft</code>	Accumulated movement
<code>self._last_path_hash</code>	To detect changed paths

Design Intent:

- Ensure simulator returns to a clean baseline after finishing a route.
- **Does NOT send any PUT requests**, preventing unwanted frontend resets.

6. Direction Handling

`_normalize_dir_value(val)`

Standardizes direction input.

Accepts:

- "START" or "STOP" strings
- {"directions": "..."}
- {"command": "..."}
- Anything else → "STOP"

Used by `get_directions()` .



`async def get_directions()`

GETs `/directions` and returns "START" or "STOP" .

Robustness features:

- Fallback to "STOP" on errors
- Attempts JSON parse if body begins with "{"



`async def put_directions_stop()`

Sends:

```
{"command": "STOP"}
```

JSON

to `/directions` when the route has ended.

Design intent:

This intentionally triggers the backend's emergency stop/broadcast behavior.



7. Path Loading

`async def load_path_if_changed()`

Steps:

1. GET `/grid/path`
2. Compute SHA-256 of response
3. If unchanged → return early
4. Parse JSON into point list
5. Convert `"r"` and `"c"` to floats
6. Update:
 - `self.path`
 - `self.curr` = first point
 - `self.next_idx` = 1 if ≥ 2 points else 0
7. Print polyline length

Design Intent:

- Avoid unnecessary reloads and simulator jumps
- Ensures simulator starts **exactly positioned on the first waypoint**

↪

8. Distance Tracking

`distance_remaining_ft()`

Computes full remaining distance from current position to final waypoint.

Uses `segment_length_feet()` for continuous accuracy.

Used when constructing `/current-values` payloads.



9. Status Updates

```
async def put_status(...)
```

Builds a JSON payload containing:

Position & Velocity

- `X_ECI` , `Y_ECI` (converted to feet)
- `Vx_ECI` , `Vy_ECI` = always `2 ft/s`

Robot State

- `Heading` (text label)
- `isMoving`
- `SequenceNum` (waypoint number 1-based)
- `errorCode` , cable flags

Telemetry

- `cableRemaining`
- `percentBatteryRemaining`
- `distanceTraveled`
- `distanceRemaining`

Only sent:

- On `"START"` heartbeat

- On waypoint arrival
- On STOP command received

Design Intent:

Minimize traffic; simulator should NOT spam status during every tick.

10. Main Loop Logic

```
async def run()
```

10.1 Session Loop

A new `aiohttp.ClientSession` is created for each outer loop cycle.

10.2 WAIT PHASE

Simulator prints:

```
// "Waiting for START and a non-empty /grid/path..."
```

Loop until:

- `/directions` returns `"START"`
- path is loaded and non-empty

On START:

- `self.running = True`

- Sends a PUT to `/current-values` (START heartbeat)

⌘

10.3 MOVEMENT PHASE

Runs until:

- STOP is received, **or**
- route finishes, **or**
- path is modified (reloaded)

STOP Edge Behavior:

If previously running and now `"STOP"` :

- Sends PUT with `isMoving=False`
- Does **not** reset state

⌘

10.4 Completion Logic

If:

```
self.next_idx >= len(self.path)
```

Then:

1. Print route completion
2. PUT `{"command": "STOP"}` to `/directions`
3. `reset_state()`
4. Break back to WAIT phase

3

10.5 Movement Tick

If actively running:

- Compute distance to next waypoint
- Compute how far robot can move this tick
 - `step_ft = SPEED_FTPS * POLL_DT = 1 ft per tick`

Two cases:

A. Full Arrival Within This Tick

- Teleport exactly to target waypoint
- `distance_traveled_ft += moved_ft`
- `cable_remaining_ft -= moved_ft`
- `percent_batt -= 1`
- `next_idx += 1`
- PUT waypoint arrival

B. Partial Movement Toward Target

Interpolate fractional `(r, c)` using:

```
ratio = step_ft / remaining_ft
```

Updates:

- `self.curr`
- `distance_traveled_ft`
- `cable_remaining_ft`

11. Program Entrypoint

```
if __name__ == "__main__":  
    asyncio.run(Simulator().run())
```

Runs the simulator forever until Keyboard Interrupt.

